K.J Somaiya School of Engineering

Department of IT (AI & DS)

Division B | Batch B3 | Group 8

Academic year 2026–2027

GenAI Powered Business Intelligence and Strategic Insight Generation System

Insight BI Application Project Report for Group 8

Submitted by Group 8

Team member	Roll number
Vruddhi Mule	16014223099
Rajdeep Pandey	16014223064
Sagar Jadhav	16014223070

Mini project in generative artificial intelligence and business analytics

October 2026

Abstract and report overview

Abstract

Insight BI is a web application that transforms structured sales data into business intelligence and strategic explanations. The project addresses the effort required to move from spreadsheet records to an understandable view of revenue, cost, profitability, and performance across business segments. Users can load a sample dataset or upload a CSV, examine a dashboard, ask questions about the selected records, and evaluate business scenarios.

The system combines deterministic numerical analysis with an optional generative AI layer. A shared TypeScript module computes all business metrics. The server validates the selected records again before sending calculated summaries and a question to the Gemini API. This approach grounds generated explanations in explicit numerical evidence. Without a configured API key, the application uses clearly identified rule-based demo responses, allowing the complete demonstration workflow to run without external model access.

The implemented prototype includes CSV validation, regional and date filtering, visual summaries, natural-language questions, scenario calculations, and report exports. Fourteen automated tests verify the core calculations and provider contract. Browser checks verify the main user interactions. Live generative output quality has not been evaluated because no provider key was configured during verification. The application is an educational decision-support prototype, not a validated forecasting or autonomous decision-making system.

Report contents

Section	Page
Introduction and objectives	3
Requirements and data model	4
System architecture	5
Analytical methodology	6
Implementation and AI integration	7
Application dashboard	8
Insight generation interface	9
Strategy planning interface	10
Testing and evaluation	11
Deployment and limitations	12
Conclusion and references	13

Introduction and objectives

Project context

Business data often arrives as rows of dates, categories, locations, revenues, costs, and orders. A spreadsheet can hold these values, but interpretation still requires consistent formulas, segment comparisons, and clear communication. Non-specialist users may struggle to connect a changing revenue chart with a practical next step. The project provides a single workflow for exploring data, explaining observations, and testing assumptions.

Problem statement

Develop an accessible application that accepts structured business data, computes reproducible performance indicators, explains relevant patterns in natural language, and helps users compare simple strategic scenarios. The interface must distinguish measured facts from suggestions and distinguish actual language-model responses from a rule-based demonstration.

Objectives

- Validate CSV inputs before analysis and preserve the previous dataset when an upload is rejected.
- Calculate revenue, recorded cost, gross profit, gross margin, orders, and average order value consistently.
- Visualize monthly performance and category contributions with region and date filters.
- Generate explanations from calculated summaries through an optional Gemini integration.
- Provide a rule-based demonstration path when no provider key is configured.
- Compare price, volume, and unit-cost assumptions without presenting them as predictions.
- Make the application reproducible through documented setup, tests, source code, and an academic report.

Scope

The prototype analyzes a single dataset in Indian rupees and supports up to 10,000 records. It does not include transactional databases, automatic enterprise connectors, user-created accounts, historical report storage, statistical forecasting, or a custom trained model. These boundaries keep the mini project focused on a complete and explainable analytical workflow.

Requirements and data model

Software and execution requirements

Local execution requires Node.js 22.13 or newer, npm, and a modern browser. React and TypeScript implement the interface; Vinext and Vite provide the application build and development server. Server routes run in a Cloudflare Workers-compatible environment. Gemini access is optional and requires an API key and a supported model in the provider account. No application database is required.

CSV schema

Field	Type	Validation or meaning
date	Date text	A real YYYY-MM-DD date between 1900 and 2100
category	Text	Required business category, 1–80 characters
region	Text	Required geographic segment, 1–80 characters
channel	Text	Optional column; absent values default to Direct
revenue	Number	Non-negative amount in Indian rupees
cost	Number	Non-negative cost using a consistent accounting definition
orders	Integer	Non-negative order count summed across records

Data quality controls

The parser supports quoted fields, embedded commas, escaped quotation marks, UTF-8 byte-order marks, and common line endings. It rejects missing or duplicate headers, inconsistent column counts, malformed quoting, invalid dates, negative numeric values, blank required numbers, and fractional order counts. Uploads are limited to 2 MB and 10,000 rows. Each grouping dimension is limited to 50 distinct values; date coverage is limited to 3,660 days. Each numeric value is capped at one trillion.

An entire invalid upload is rejected rather than silently dropping rows. Duplicate rows are retained because identical aggregate records may be intentional. Users are responsible for ensuring that repeated records do not double-count business activity. A row may represent one transaction or a sales aggregate, so the application uses the orders column rather than equating row count with order count.

Demonstration dataset

The bundled dataset is generated deterministically and contains 288 synthetic records for January–June 2025. It covers four product categories, four Indian regional labels, and three sales channels. It is designed for demonstrations and testing and does not describe an actual business.

System architecture

The system separates data presentation, deterministic analytics, and optional language generation. A shared analytics module is used in both the browser and the server so the numerical definitions remain consistent.

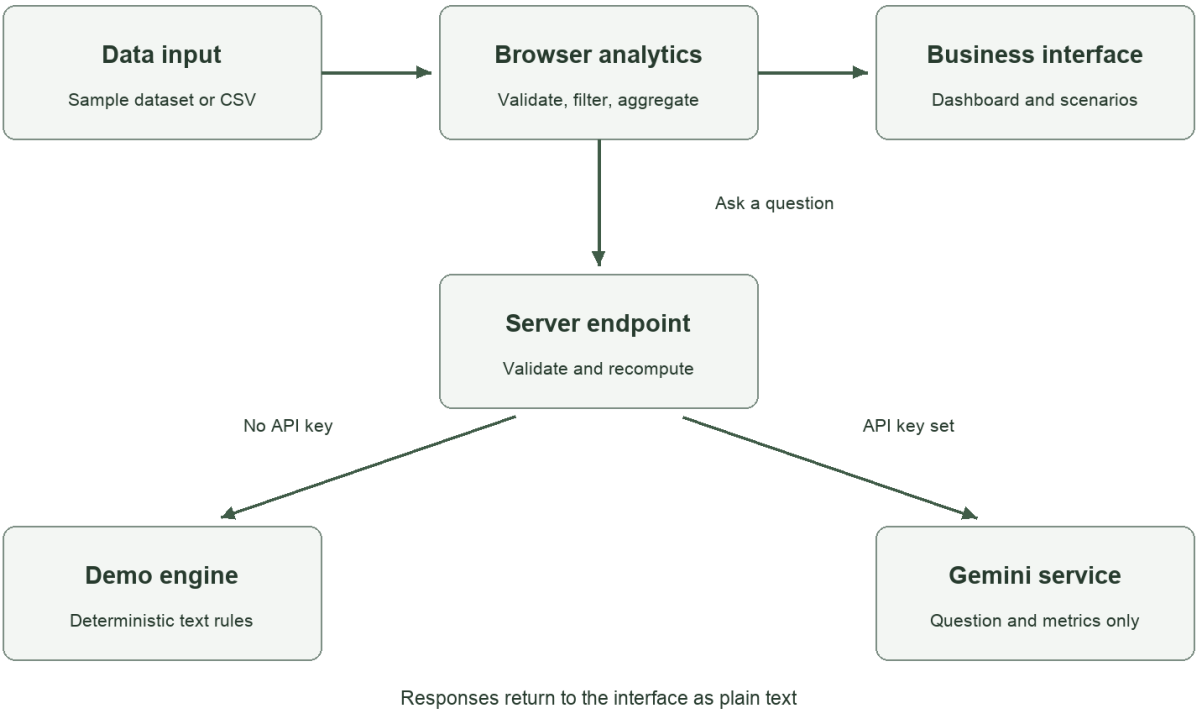


Figure 1 Data flow and model selection in Insight BI

Browser layer

React holds the current dataset, selection filters, search text, and scenario inputs in memory. The analytics module derives filtered totals and grouped data. The dashboard and strategy lab update from that same selection. A refresh restores the sample dataset, which avoids leaving persistent uploaded business records on the device.

Application server layer

GET /api/insights reports whether the provider key is configured. POST /api/insights accepts a question and the selected records. It checks the request size and origin, validates the payload, and recalculates the summary. It then selects the rule-based engine or Gemini integration. The API returns plain text with an explicit mode label.

External model boundary

When Gemini is enabled, only the question and calculated summaries are transmitted to the provider. Individual sales rows are not sent to the model. Aggregated labels and business values can still be sensitive, so this behavior is disclosed in the interface. API credentials remain in server environment variables.

Analytical methodology

Business performance metrics

For selected records, let R be the sum of revenue, C the sum of recorded cost, and O the sum of orders. The application uses the following definitions.

Metric	Formula
Gross profit	R − C
Gross margin	100 × (R − C) / R
Average order value	R / O
Category revenue share	100 × category revenue / R
Latest month growth	100 × (latest revenue − prior revenue) / prior revenue

Zero revenue and order denominators are displayed as zero to keep the interface finite; these ratios should be treated as unavailable rather than as evidence of normal business performance. Growth is unavailable when the previous calendar month is absent or has zero revenue. The application does not compare nonconsecutive months as a month-on-month growth rate. A partial month may still be present, and this limitation is shown near the chart and in the insights.

Strategic scenario calculation

Let p, v, and c be price, volume, and unit-cost changes expressed as fractions. The scenario assumes that all recorded costs vary with volume. The calculated scenario values are:

Scenario revenue = R × (1 + p) × (1 + v)

Scenario cost = C × (1 + v) × (1 + c)

Scenario gross profit = scenario revenue − scenario cost

Profit change = scenario gross profit − current gross profit

For a small verification example with revenue 250 and cost 160, a 10% price increase, 20% volume increase, and 10% unit-cost decrease produce revenue 330, cost 172.80, and gross profit 157.20. This calculation is verified by an automated test. Resetting all assumptions to zero reproduces the original profit of 90.

Rule-based observations

Insight cards identify the highest-revenue category, the category with the lowest gross margin, and the latest month comparison. Recommended actions are presented as checks or experiments, such as reviewing costs or validating demand. The application does not infer causal reasons from aggregate correlations.

Implementation and AI integration

Application modules

Module	Implementation responsibility
app/page.tsx	Navigation, dashboard, data explorer, insight questions, strategy controls, and project guide
lib/analytics.ts	CSV parser, validation, synthetic sample generator, filtering, grouping, scenario math, and demo answers
lib/ai.ts	Provider request construction, model selection, response parsing, and error handling
app/api/insights/route.ts	HTTP boundary, server environment configuration, request checks, and responses
app/globals.css	Responsive layout, visual states, keyboard focus, and reduced-motion styling

Prompt construction and grounding

The backend builds a JSON context containing the question and computed summary. A system instruction requires the model to use supplied metrics, identify relevant periods and metric names, avoid invented values or causal claims, distinguish observations from hypotheses, and acknowledge missing information. Currency is INR, and profit is explicitly defined as gross profit from the cost column. Temperature is set to 0.25 to encourage restrained responses; this does not guarantee factual correctness.

Dataset labels and questions are designated as untrusted data in the prompt. No model-generated SQL, JavaScript, or HTML is executed. Returned text is rendered by React as text. There is no retrieval database or embedding index; the grounding context consists of numerical aggregates, not retrieved documents.

Modes and failure handling

Without GEMINI_API_KEY, the server uses deterministic keyword routes for categories, regions, channels, margins, trends, and summary questions. Unsupported demo questions explain the limitation. With a key, the server calls the Gemini Generate Content endpoint using GEMINI_MODEL. Provider errors, empty answers, and timeouts produce visible errors rather than being silently relabeled as successful AI output. Requests have a 30-second provider timeout and a 40-second browser timeout.

Data handling

Uploaded rows remain in browser memory until an insight question is submitted, at which point the selected records reach this application server for validation. The server forwards only aggregates to Gemini. Credentials are excluded from the frontend and ignored by Git. CSV exports protect formula-like text labels. The public demo uses rule-based responses without a provider key. Same-origin checks are implemented; production-grade quotas and per-user authorization remain outside the prototype.

Application dashboard

The overview combines performance cards, a monthly revenue and gross-profit chart, category contribution, and three rule-based insight cards. Region and date selections affect the same dataset used by the insight service.

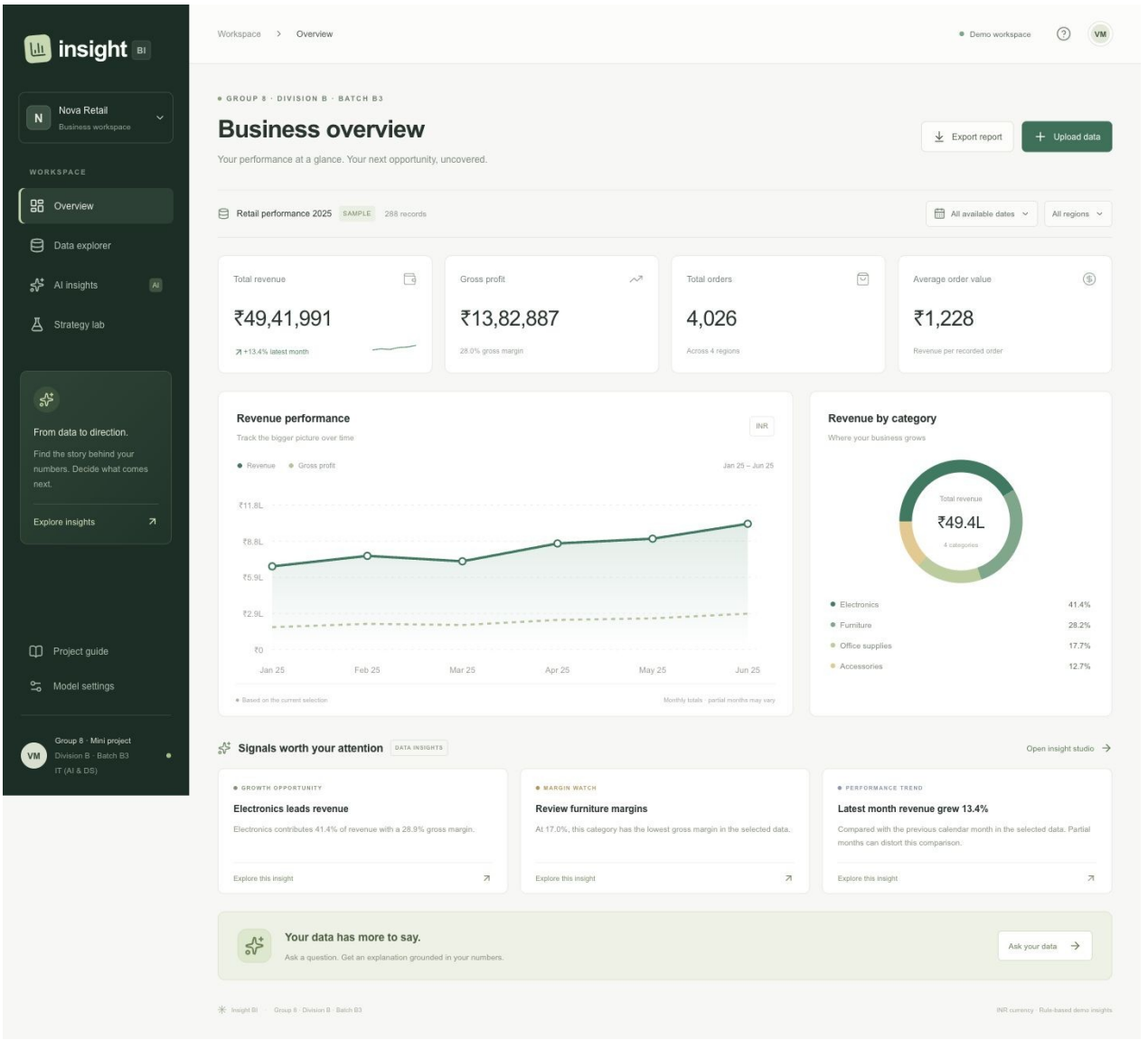


Figure 2 Desktop dashboard using the complete synthetic sample dataset

The sample produces revenue of INR 4,941,991, recorded cost of INR 3,559,104, gross profit of INR 1,382,887, and 4,026 orders. The gross margin is approximately 27.98%, and average order value is approximately INR 1,227.52. Electronics contributes INR 2,043,888 of revenue. These values are calculated from the bundled sample, not from live business records.

Insight generation interface

The insight studio provides suggested questions, a free-text input, and the current dataset context. The response label tells the user whether the answer came from Gemini or the rule-based demonstration. Changing the dataset or filters clears a previous answer so it is not presented as belonging to a new selection.

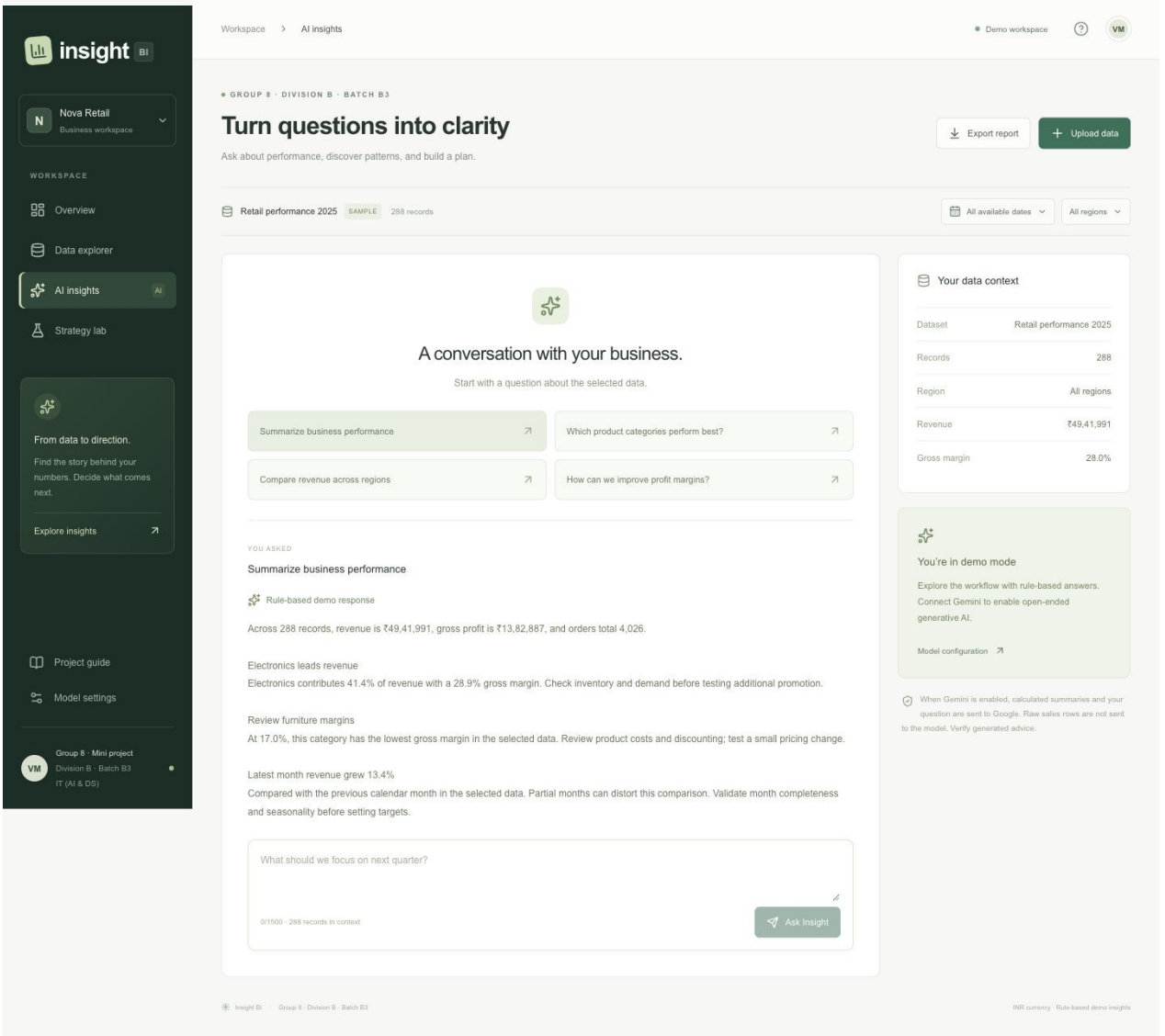


Figure 3 A completed rule-based summary in the insight studio

The demonstrated response reports the sample revenue, profit, and order totals, then describes category contribution, margin review, and monthly revenue change. The displayed demo-mode notice is intentional. The project implements a real provider integration, but does not claim that this screenshot shows a live language-model response.

Strategy planning interface

The strategy lab lets users change average selling price, sales volume, and cost per unit. Results are recalculated immediately. The panel shows scenario revenue, cost, gross profit, margin, and the difference from the selected baseline. Exporting the report records both assumptions and results.

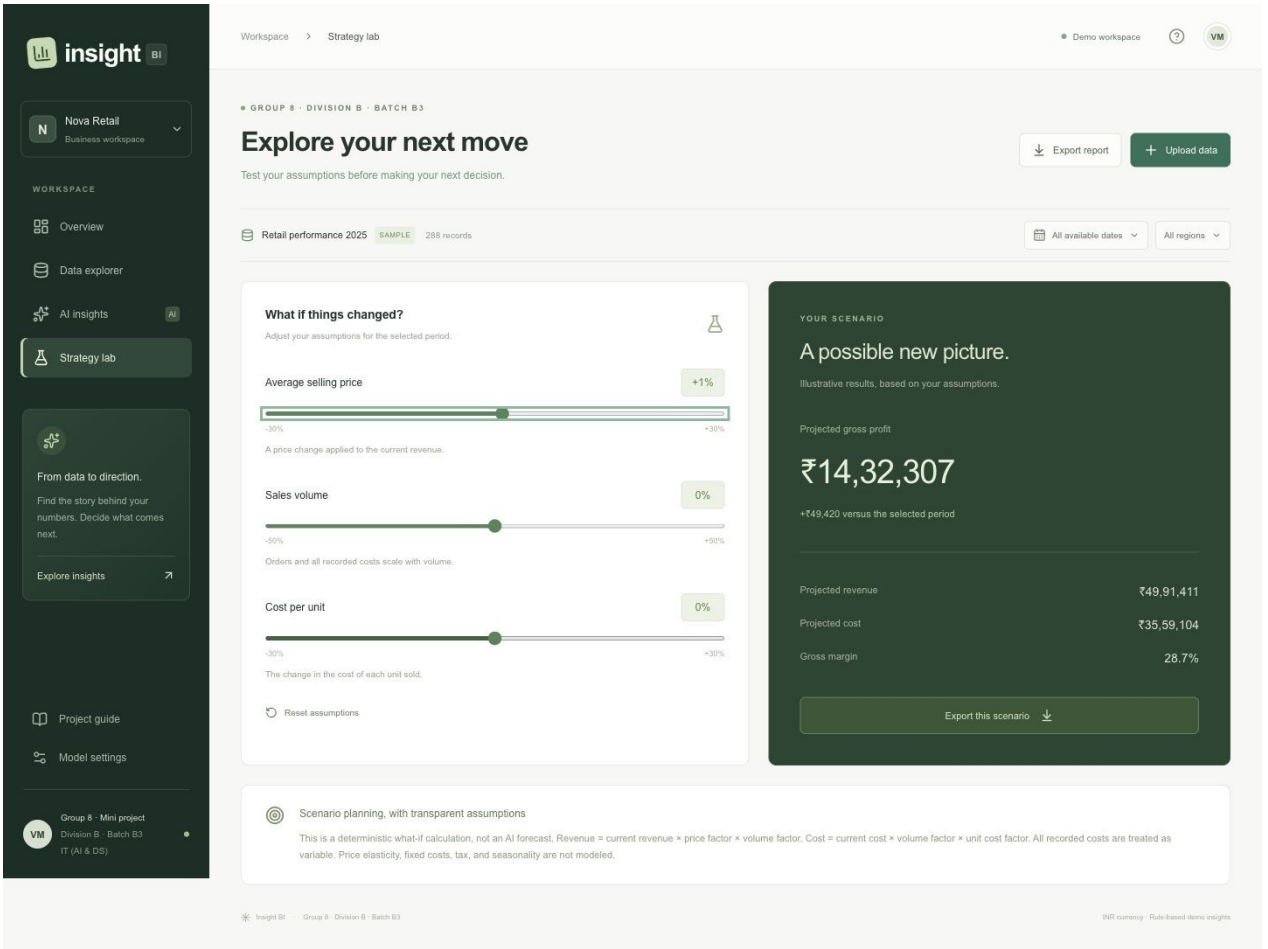


Figure 4 Strategy lab with a one percent price increase and unchanged volume and unit cost

For the sample baseline, increasing only price by 1% gives revenue of INR 4,991,410.91 and gross profit of INR 1,432,306.91. The cost stays INR 3,559,104 because the volume and unit-cost assumptions are unchanged. The interface rounds currency to whole rupees. Resetting the controls returns the original values.

Interpretation

The output is a conditional calculation. It does not establish that customers will accept a price increase or that order volume will remain unchanged. Fixed costs, capacity limits, demand elasticity, and competitive responses are not included. The results should guide further investigation rather than be treated as a forecast.

Testing and evaluation

The project includes 14 automated tests implemented with the Node.js test runner. The tests use fixed datasets and explicit expected values so that a calculation error produces a reproducible failure. The provider integration is tested using mock HTTP responses. The automated suite and TypeScript checks passed locally, and the production build completed.

Test group	Coverage	Result
Core metrics	Revenue, cost, profit, margin, AOV, ranking, and month growth	Pass
Selection filters	Region and calendar-month filtering relative to dataset dates	Pass
CSV input	Round trip, quoted commas, escaped quotes, missing/duplicate headers, malformed quotes	Pass
Validation	Impossible dates, blanks, negatives, fractional orders, row and category limits	Pass
Edge cases	Zero denominators, empty summaries, and missing calendar months	Pass
Scenarios	Known arithmetic and exact restoration of baseline assumptions	Pass
Export handling	Formula-like labels escaped in CSV output	Pass
AI contract	Grounded demo text, aggregate-only provider payload, invalid requests, provider errors, empty output	Pass

Browser verification

Manual browser checks confirmed that selecting West updates revenue to INR 1,520,502 across 72 sample records; the CSV sample uploads with 288 valid records; searching Furniture finds 72 records; the summary question returns a labeled demo response; scenario adjustment and reset update the values; and a business report downloads. Desktop and mobile layouts were inspected. A server/client hydration mismatch found during development was corrected before the final verification.

Evaluation limits

The tests establish selected numerical and interaction behaviors. They do not establish real-world business usefulness, statistical forecasting accuracy, or language-model reliability. No live Gemini call, user study, large-scale load test, or production security audit was performed. An appropriate next evaluation would use a separately checked business dataset and human ratings of factual support, completeness, and actionability of generated answers.

Deployment and limitations

Local installation

- Install Node.js 22.13 or newer and npm.
- Clone https://github.com/vru9/genai-business-intelligence and enter the project directory.
- Run npm ci, followed by npm run dev. Open the local URL printed in the terminal.
- For optional generative mode, copy .dev.vars.example to .dev.vars, set GEMINI_API_KEY and GEMINI_MODEL, and restart.
- Run npm test and npm run typecheck to verify the project; npm run build creates the deployment build.

Repository and hosted access

The GitHub repository is public so faculty can inspect the source, sample data, tests, screenshots, and this report. The hosted application uses the Sites platform and is public, so anyone with its link can open the demo. Faculty can also run the repository locally. The public demo works without a provider key using clearly labeled rule-based responses. API keys are not included in the repository.

Repository: https://github.com/vru9/genai-business-intelligence
Hosted address: https://insight-bi-vruddhi.rajdeepvp273.chatgpt.site

Current limitations

- Uploads and answers are session-memory state and are not saved across refreshes.
- The schema uses one currency and does not support negative sales or returns.
- The recorded cost definition determines the meaning of profit; taxes and omitted expenses are not inferred.
- Monthly comparisons may contain incomplete months and are not adjusted for seasonality.
- The scenario model treats every recorded cost as variable and omits price elasticity and fixed expenses.
- Demo answers follow limited keyword rules; actual generative answers depend on an external model, credentials, quota, and network availability.
- Prompt grounding reduces avoidable errors but cannot guarantee that a model follows every instruction.

Suggested demonstration

Show the dashboard, apply a regional filter, upload the sample CSV, search a category, ask a summary question, explain the mode label, adjust a scenario, reset it, and export the business report. Finish by showing the test suite and explaining the difference between an observation, a recommendation, and a conditional scenario.

Conclusion and references

Conclusion

Insight BI implements an end-to-end educational workflow for structured business analysis: importing data, validating it, computing metrics, visualizing performance, generating explanations, and exploring strategic assumptions. Its main design choice is to calculate numerical facts before asking a language model to explain them. The same analytics module underpins both the dashboard and the server-side context, making the metrics easier to verify.

The project is suitable for demonstrating practical integration of web development, data analysis, prompt construction, and API-based generative AI. A working rule-based mode makes the application demonstrable without credentials, while explicit labels prevent that mode from being confused with actual generative output. The test suite and documented assumptions support a more transparent presentation of both the capabilities and limits of the prototype.

Future enhancements

- Add persistent datasets and saved reports with per-user access controls.
- Introduce configurable column mapping, multiple currencies, returns handling, and richer data quality diagnostics.
- Evaluate live model responses against a human-reviewed set of business questions and expected facts.
- Add statistical forecasting only with backtesting, error measures, and uncertainty estimates.
- Extend scenarios with fixed and variable cost separation, demand elasticity, and sensitivity analysis.
- Add request quotas, audit trails, and cost monitoring before wider production use.

References

1. Google AI for Developers. Gemini Generate Content API. https://ai.google.dev/api/generate-content

2. React. Learning React and component state. https://react.dev/learn

3. Microsoft. TypeScript Handbook. https://www.typescriptlang.org/docs/handbook/intro.html

4. Cloudflare. Cloudflare Workers documentation. https://developers.cloudflare.com/workers/

5. Node.js. Test runner documentation. https://nodejs.org/api/test.html

6. Project source. Insight BI repository and test suite. https://github.com/vru9/genai-business-intelligence

Report prepared for academic year 2026–2027. Application and test evidence recorded on 16 October 2026.